SERVICE-ORIENTED ARCHITECTURE (SOA) GOVERNANCE IMPLEMENTATION SUPPLEMENTAL GUIDE

The materials presented in this technical guideline provide specific supplemental information into OUM for the purposes of SOA Governance Implementations work. The Service-Oriented Architecture (SOA) Governance Implementation Supplemental Guide contains the following components:

SOA Governance Implementation Overview

This component provides an overview of the SOA Governance Implementation.

SOA Governance Implementation Supplemental Task Guidelines

The Supplemental Task Guidelines should be used in conjunction with the standard OUM tasks in order to accomplish SOA Governance Implementation work at the detail task level.

PDF View

You may also prefer to access this information through the **Service-Oriented Architecture (SOA) Governance Implementation Supplemental Guide (PDF View)** located in the Supplemental Guides section of the Supplemental Guidance page.

Back to Top

PURPOSE

The purpose of this document is to provide an overview of SOA Governance Implementations solutions, and to provide implementation teams with *supplemental guidance* for the successful planning and execution of SOA Governance Implementations.

Back to Top

AUDIENCE

This document is intended for practitioners who will be designing and implementing SOA Governance Implementation solutions based on the OUM (Oracle Unified Method) approach.

Back to Top

HOW TO USE THIS DOCUMENT

This document contains supplemental guidance for planning and executing SOA Governance Implementations and should be used in conjunction with and not instead of the OUM base method content.

Back to Top

LIMITATIONS TO THIS DOCUMENT

This document does not describe OUM in detail – the reader must review the documentation and training materials referenced below to gain a basic understanding of each.

Back to Top

RELATED DOCUMENTATION AND TRAINING

The following related documentation & training provide additional detailed guidance related to OUM implementations and are prerequisites to this document:

- OUM Method Pack – Desktop Installable or Hosted version
- OUM Training and Accreditation Program

Back to Top

CONTRIBUTORS

Sigrid Gylseth, Steve Verba, Tomas Vitvar

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

SERVICE-ORIENTED ARCHITECTURE (SOA) GOVERNANCE IMPLEMENTATION OVERVIEW

WHAT IS SOA GOVERNANCE IMPLEMENTATION?

The SOA Governance Implementation Supplemental Guidelines provides detail information on processes and tasks that an organization can follow to implement the SOA Governance Model. When you develop SOA Governance in your organization, depending on your organization's approach and a level of SOA maturity, you may follow a strategic approach or a pragmatic approach.

In a strategic approach, you first perform SOA Governance planning, such as an enterprise-wide analysis of Governance Structures and Practices, develop a Governance Vision and create a Governance Roadmap. After that, you perform SOA Governance Implementation in which you design and develop structures of your enterprise's and projects' assets and asset lifecycle management, and implement a SOA Governance tooling support, such as, an Enterprise Repository and a Service Registry. You may find details on SOA Governance Planning in the SOA Governance Planning view and the SOA Governance Planning Supplemental Guide.

In a pragmatic approach, your organization already has a certain level of SOA Governance implemented. For example, the organization uses Excel spreadsheets or a wiki to maintain assets (e.g., services, applications) and a lifecycle process around these assets implemented manually or with email support. Further, the organization might only apply the governance in a small number of projects while there are many projects that do not have any level of governance at all. In this approach, you can follow the tasks of the SOA Governance Implementation guide. You may also take into account tasks from the SOA Governance Planning view and determine which tasks you need to execute in order to fill gaps in your enterprise-wide SOA Governance Model.

The goal of SOA Governance Implementation is to move from the current state of governance to a state where all SOA projects properly follow and fully adopt the SOA Governance Model. The future state can be characterized as follows:

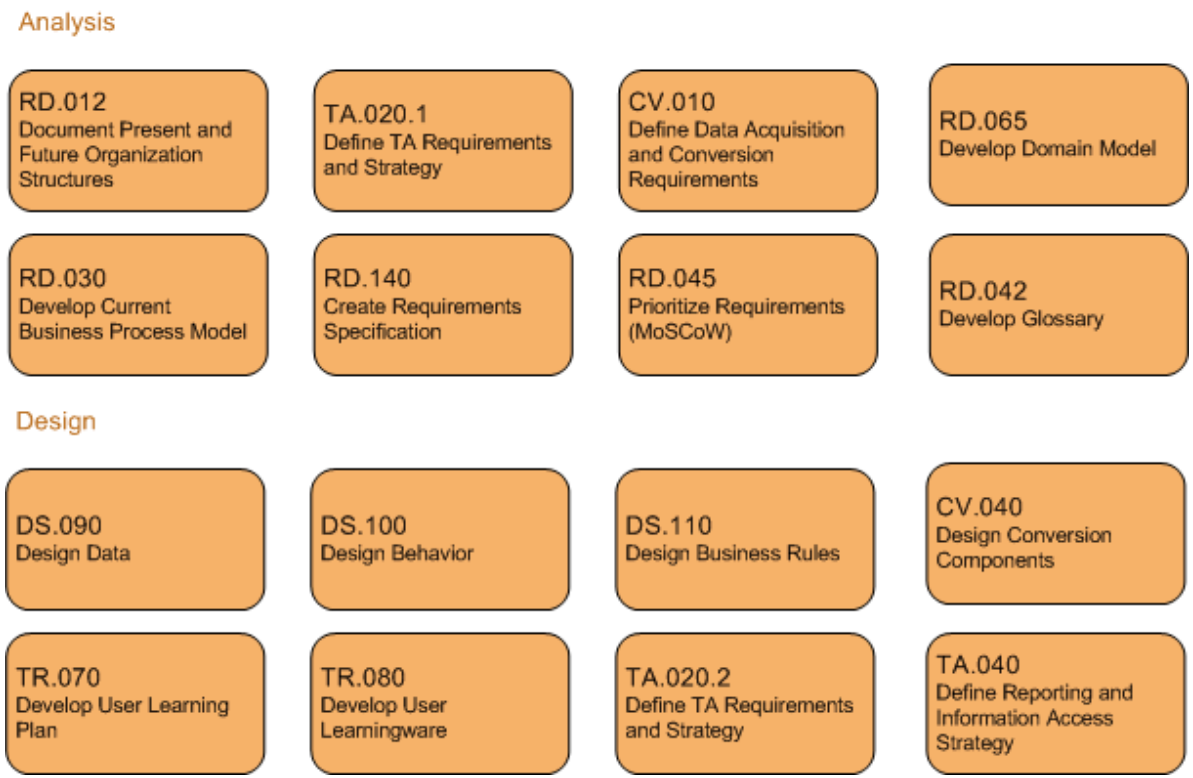
1. There is central asset management in place, that is, an Enterprise Repository is implemented and up and running,
2. All existing assets from legacy repositories, applications and environments are in the repository and they have been validated against the running environment.
3. There is a service lifecycle defined and incorporated in software and application development processes that produce or consume assets.
4. All staff members of the organization are educated on SOA Governance, that is – they are aware of the service lifecycle and apply it in their daily job.

Back to Top

OVERVIEW

This supplemental guide describes the methodology that you can follow to create a SOA Governance specification and develop SOA Governance tooling support for an organization that already has several SOA projects running or implemented. Moreover, the organization already may have a certain level of SOA Governance in place, however, it is not well documented, structured and implemented. Typically, this type of organization started to develop application integration in SOA style – that is, the organization uses Enterprise Service Bus to integrate applications, however, there is a lack of standards, guidelines and structures to govern SOA projects. Moreover, the organization still spends too much effort on maintaining ad hoc application integration that typically happens in a point-to-point style while the organization's goal is to gradually move from the point-to-point integration style to SOA. For purposes of reporting and a general overview of services and applications in the organization's environment, there is a repository, usually an Excel spreadsheet and a manual process that the organization executes to gather information and maintain the repository.

The organization decides to implement SOA Governance to support existing and future SOA projects. For this purpose, the organization will use an Enterprise Repository that provides tooling support for SOA Governance. The Enterprise Repository should also provide a certain level of flexibility to accommodate the organization's specific needs, such as, customizations of asset types and customizations of a service lifecycle. This supplemental guide describes a development process to migrate from a legacy service repository to a fully-fledged Enterprise Repository and a service lifecycle that the organization should follow when developing any new services for application integration. The following figure shows an overview of the OUM tasks that are included in this supplemental guide with additional guidance for this development process.



When you have performed SOA Governance Planning prior to a SOA Governance Implementation, the following table shows how you the work products from SOA Governance Planning can be used as prerequisites or touch points to a SOA Governance Implementation:

Envision Governance Task	Touch Point to SOA Governance Implementation Task	Usage (if GV Work Product is Available)
Determine Organizational Impact of Governance (GV.040)	Document Present and Future Organization Structures (RD.012)	Review the organizational changes that have been decided upon.
GV.060 Define Policy Implementation Tooling (GV.060)	Create System Context Diagram (RD.005)	Review whether there are any policy implementation tooling decisions that impact the system context.

Determine Projects Meta Data Descriptions (GV.092) **Determine Applications Meta Data Description (GV.094)** **Determine Services Meta Data Description (GV.096)** **Determine Other Meta Data Descriptions (GV.098)**	Define Data Acquisition and Conversion Requirements (CV.010) Develop Domain Model (RD.065)	Review the Meta Data Descriptions to understand the asset properties, relationships and life cycles.
Review Customer Software Development Process (EA.095) **Define Policy Implementation Processes (GV.050.1)**	Develop Current Business Process Model (RD.030)	Review the documented "Customer Software Development Process", and what policy implementation processes have been defined.
Discover or Define Policies (GV.030)	Design Business Rules (DS.110)	Use the Governance Policy Catalog as an input to identify / design required business rules.

Back to Top

GLOSSARY

This Glossary contains important terms that are used in the SOA Governance Implementation Supplemental Guide. You can also use these terms as a starting point for creating your own SOA Governance Project Glossary as defined by task, Develop Glossary (RD.042) in the SOA Governance Implementation Supplemental Task Guidelines.

Term	Definition
Advanced Submitter	A user that defines the responsibility of a user who has editable rights for assets in an Enterprise Repository.
Asset	Anything that is of value to SOA Governance, such as, service, application, environment, framework, pattern, etc.
Asset Type	Meta data structures for a certain type of an asset.
Built-In Process	A process that supports operation of an Enterprise Repository.
Conversion	A process that transforms structures of assets from a legacy repository to new structures defined by an Enterprise Repository.
Enterprise Service Bus	A middleware that facilitates integration process running across two or more back-end applications.
ESB	See Enterprise Service Bus.
Harvesting	A process that fetches information about assets (e.g., services, process, components, etc.) from a running middleware platform to an Enterprise Repository.
Migration	A process that migrates assets from a legacy repository to an Enterprise Repository. It usually includes automated conversion and manual resolution of conflicts, errors or warnings.
Offline Service Repository	A service repository with a very limited functionality. It is usually implemented in files or spreadsheets with Governance processes implemented manually.
OOTB	See Out of the Box.

Out of the Box	A functionality of a software product that comes with the product.
Registrar	A user role that defines responsibility for an Enterprise Repository power user.
Service Lifecycle Process	A process that defines a development of services. Its states typically are: propose, plan, build, test, deploy, and retire.
Service Repository	See Enterprise Repository.
Service Registry	A software that stores information about services and their classifications and allows for searching services. It usually implements UDDI interface.
Viewer	A user role that defines read-only rights for an Enterprise Repository user.
UDDI	Universal Description, Discovery and Integration; An interface for a service registry standardized by OASIS.

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.

Oracle
Unified Method

Cloud
Home Page

Service-Oriented Architecture (SOA) OUM Governance Implementation Supplemental Guide

Resources | PDF View | Method Navigation | Current Page Navigation

SERVICE-ORIENTED ARCHITECTURE (SOA) GOVERNANCE IMPLEMENTATION SUPPLEMENTAL TASK GUIDELINES

This component contains OUM supplemental task guidance for SOA Governance Implementation.

- The second column of the following table provides a complete list of all tasks in this document for which "supplemental" guidelines are provided as well as a quick link to the "supplemental" task guideline in this document.
- The third column provides a link to the OUM Task Overview.
- The last columnprovides a link to the associated template if one is available.

The task guidelines in this document should be used to supplement the guidelines that appear in the OUM Task Overview.

OUM ID	SOA Governance Implemention Supplemental Guidance	OUM Guidance	Template
TG.SGIA	**SOA Governance Implemenation Analysis**		
RD.012	Document Present and Future Organization Structures	Document Present and Future Organization Structures	Present and Future Organization Structures
TA.020.1	Define Technical Architecture Requirements and Strategy	Define Technical Architecture Requirements and Strategy	Technical Architecture Requirements and Strategy
CV.010	Define Data Acquisition and Conversion Requirements	Define Data Acquisition and Conversion Requirements	Data Acquisition and Conversion Requirements
RD.065	Develop Domain Model (Business Entities)	Develop Domain Model (Business Entities)	Domain Model
RD.030	Develop Current Business Process Model	Develop Current Business Process Model	Current Process Model
RD.140	Create Requirements Specification	Create Requirements Specification	Requirements Specification
RD.045	Prioritize Requirements (MoSCoW)	Prioritize Requirements (MoSCoW)	MoSCoW List-Excel MoSCoW List-Word Generic Workshop Notes Generic Workshop Schedule and Workshop Preparation Notes
RD.042	Develop Glossary	Develop Glossary	Glossary

TG.SGID	SOA Governance Implemenation Design		
DS.090	Design Data	Design Data	Refer to the Task Overview for guidance.
DS.100	Design Behavior	Design Behavior	Refer to the Task Overview for guidance.
DS.110	Design Business Rules	Design Business Rules	Refer to the Task Overview for guidance.
CV.040	Design Conversion Components (Initial Load)	Design Conversion Components (Initial Load)	Conversion Component Designs (Initial Load)
TR.070	Develop User Learning Plan	Develop User Learning Plan	User Learning Plan
TR.080	Develop User Learningware	Develop User Learningware	User Learningware
TA.020.2	Define Technical Architecture Requirements and Strategy	Define Technical Architecture Requirements and Strategy	Technical Architecture Requirements and Strategy
TA.040	Define Reporting and Information Access Strategy	Define Reporting and Information Access Strategy	Reporting and Information Access Strategy

Back to Top

TG.SGIA SOA GOVERNANCE IMPLEMENTATION ANALYSIS GUIDANCE

RD.012 Document Present and Future Organization Structures

In this task, you focus on documenting present organization structures – that is, all departments, roles and external organizations involved in development processes of applications and integration middleware that relate to SOA Governance. If available, use the Enterprise Diagnostic Information (ER.040).

For example, in an organization there may exist the following departments, roles and their responsibilities that relate to SOA Governance:

- **Business Analysis Department (role: Business Analyst)**
  - Perform initial analysis of business demands.
  - Perform conceptual design describing a demand for a new or updated application functionality.
- **Solution Architecture Department (role: Solution/Domain Architect)**
  - Owns majority of enterprise applications.
  - Perform conceptual design describing a demand for a new or updated application functionality.
  - Perform technical analysis and design of business requirements.
  - Define technical specification for implementation.
  - When a requirement includes an integration of two or more applications, a solution architect contacts the Integration Architecture team.
  - Decide on how to implement the requirement and who should implement it (such as, in-house development or development by an external supplier).
- **Integration Architecture Department (role: Integration Architect)**
  - Own a service repository (such as repository built on an Excel spreadsheet).
  - Own platforms for SOA integration, develops the platform in-house or coordinates development with an external supplier.
  - Analyze integration for particular requirements, data models, assesses reusability of existing services.
  - Decide a style of the integration (such as, point-to-point or SOA) when developing new services.
  - Approve new integration functionalities.
- **Enterprise Architecture Department (role: Enterprise Architect)**
  - Overview enterprise architecture design, data models and processes.
- **Testing and Deployment Department (role: Tester)**
  - Review the implementation documentation.
  - Test the developed functionality.
  - Deploy the new functionality.

External Supplier/Vendor (role: Developer, Architect, Consultant)

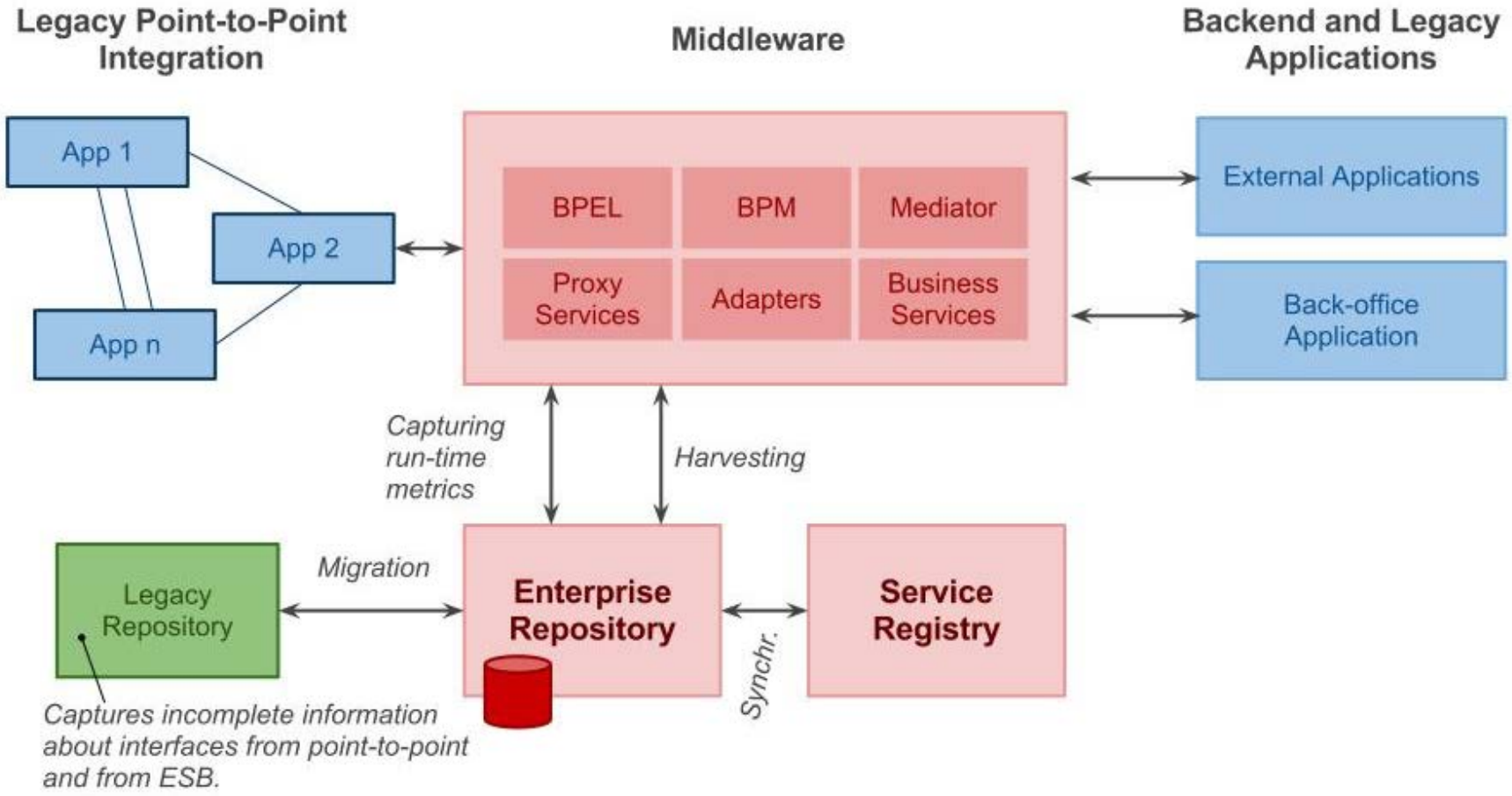
- Responsible for development of an application/integration.

Back to Top

TA.020.1 Define Technical Architecture Requirements and Strategy

In this task, you create a diagram of an overall high-level view of the organization's systems related to SOA Governance and an envisioned integration of SOA Governance tooling support. The purpose of this diagram is to gain an understanding of systems, applications or groups of applications that the SOA Governance should support.

For example, document existing major integration platforms together with their main functions and connections, major applications that either provide or consume services, integration styles such as point-to-point or enterprise service bus integration, and any existing repositories such as for documents or any other assets (services/interfaces). Such repositories may be available either online (for example, existing enterprise repository or service registry such as UDDI) and off-line (that is, manual service repository such as wiki pages, files, spreadsheets, etc.).



Back to Top

CV.010 Define Data Acquisition and Conversion Requirements

In this task, you further analyze requirements for any existing sources of information that you previously documented in the Technical Architecture Requirements and Strategy (TA.020) and that provide or contain information the organization's assets. Take into account any existing middleware platforms and online or offline service repositories or registries. For example, identify a legacy repository in an Excel spreadsheet, existing middleware platform and a service registry with following requirements:

- All information about services and applications from Excel repository should be migrated to the Enterprise Repository.
- All existing services and related artifacts from middleware platforms should be harvested to the Enterprise Repository.
- Information in the service registry should be synchronized with information in the Enterprise Repository.

For each source of information, document:

- Who is the owner of the data, such as, who is responsible for the overall repository and who is responsible for any portions of it or types of data that the repository holds.
- How you can access the data, for example, in a file, using an API/service, or a database. Any permissions/credentials you need to have to access the data.
- Validity of the information. If the information is not valid or is valid only partially, you need to figure out whether you can obtain an up-to-date information and how.
- The frequency of change of the data.
- The amount of the data.
- The data formats such as XML, natural text or any proprietary format.
- Any specific knowledge or tools/libraries you need to have to access the data programmatically. Programmatic access to the data is important for any conversion you later perform.

For example, an organization may use an Excel spreadsheet with information about services and applications as follows:

- The owner of the repository is the member of Integration Architecture department.
- There are approximately 1,000 services and 200 applications available in the spreadsheet.
- The owner irregularly sends an email to owners of applications (solution architects) and asks them to update the data about their applications and services.
- The owner sometimes does not get a response from application owners thus not all data is up-to-date.
- Since the data is in the Excel spreadsheet, you can export them to a variety of formats such as CSV. However, there are inconsistencies in the data and some data are in natural text not following strict rules.
- There are open source libraries that you can use to access the Excel spreadsheet programmatically.

Back to Top

RD.065 Develop Domain Model

In this task, you analyze and document the existing Domain Model of the organization's SOA Governance assets. First, identify assets that the organization uses, such as, services, applications, environments, etc. For each asset, identify *meta data* that describe the asset and rules that define the integrity of the data. For example, in task CV.010 you discover that the organization uses an offline service repository in an Excel spreadsheet with information about services and applications that expose and/or consume services. Further, there is a Solution Architecture Department that owns every application, and there are rules that the owner of the repository must follow in order to keep the repository's data consistent.

An example of meta data, rules and responsibilities for the service description follows. Note that this description reflects the current state of an example organization and is not necessarily the best practice for meta data description. There are two roles in this example, namely an application owner and a service owner. The application owner is a member of Solution Architecture Department (see the example in task RD.012) and the service owner is the application owner who owns the application that provides the respective service.

- **Service Name**
  - Application Owner assigns a service name when adding a new service.

    - Name is unique across all services in the repository.
    - Naming Convention is: APPNAME-IDENT where APPNAME is the name of the application that exposes the interface and IDENT is a unique identifier within the application. The identifier may be, for example, a data structure on which the service operates.
    - When the interface already exists in the list, its name cannot be changed.
- **Application Provider – name of an application that provides the service**
    - Application owner also owns a service that the application provides.
    - Name is unique across all applications in the repository.
- **Application Consumer – name of an application that consumes the service**
    - Service owner specifies the application that consumes the service.
    - Name is unique across all applications in the repository.
    - If there is more than one application consumer for the service, the application owner will add a new record (row) to the repository and copy the values of the service.
- **Data Structures – data structures for input and output messages of the interface**
    - The application owner sets the name for the data structures.
    - Data structures are dependent on the application hence same data structures across multiple applications may not mean the data structures are the same.
- **Transport – Protocol – low-level protocol for the communication with the interface**
    - The application owner sets the protocol that the service uses for communication.
    - In some cases the protocol may not be set, for example, when there are multiple protocols defined by an access mechanism.
- **WSDL – URL address of the WSDL document**
    - The application owner sets the URL WSDL address.
    - If WSDL document exist, the application owner does not provide information about data structures and the transport. It is assumed that such information is available in the WSDL's interface, binding and service descriptions.

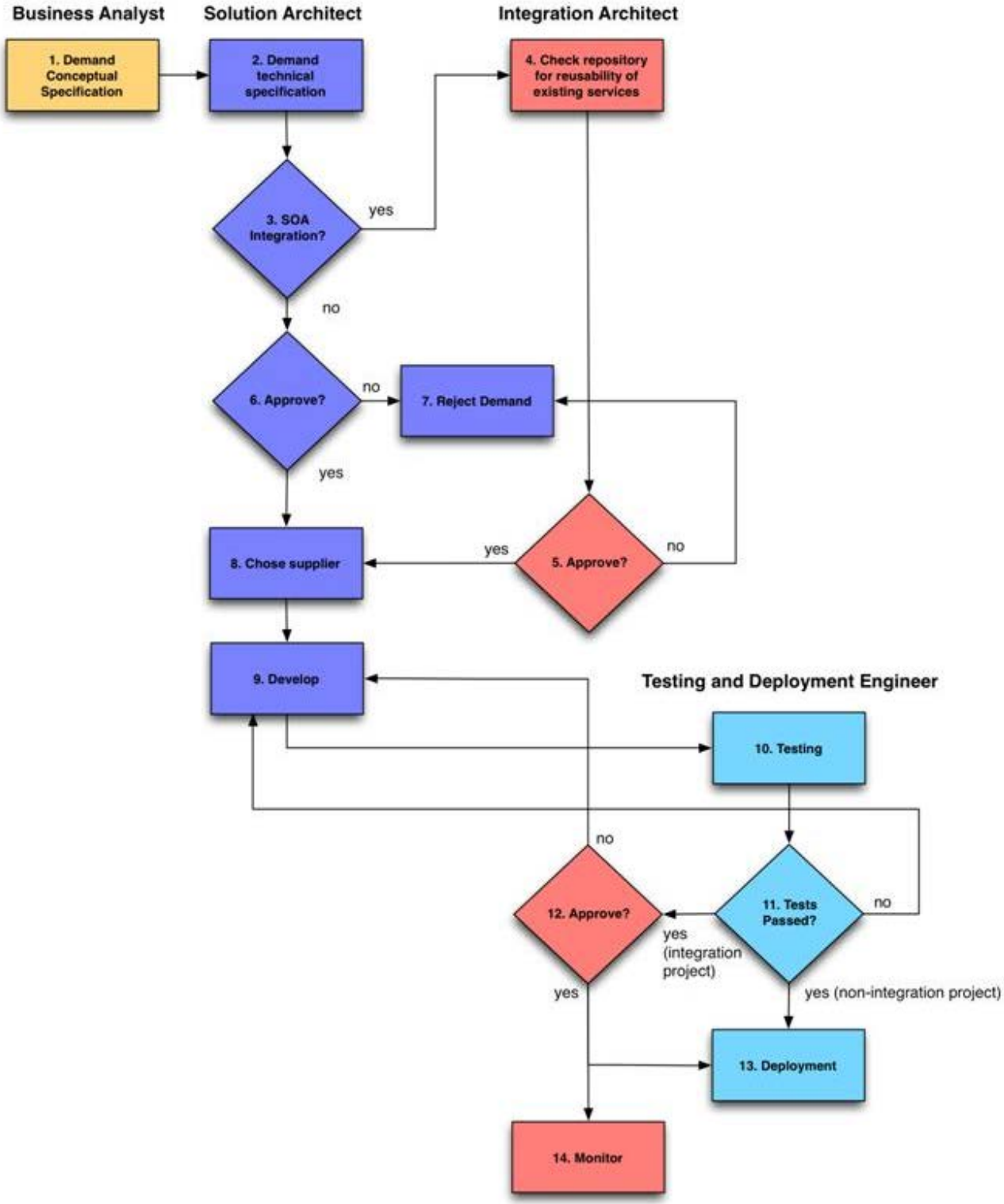
Back to Top

RD.030 Develop Current Business Process Model

In this task, you document existing process models that are related to SOA Governance. You are interested in software development processes that the organization uses for development of applications and their integrations. Such processes may produce assets that are of interest to SOA Governance such as services, applications, frameworks, environments, design-patterns etc. Moreover, there might be other processes directly related to the organization's current governance. For example, in task, Define Data Acquisition and Conversion Requirements (CV.010), you discovered that the organization uses an offline service repository and that a member of the Integration Architecture Department follows certain steps to make sure that the repository is up-to-date. In this task, you take into account the organization's structures from the Present and Future Organization Structures (RD.012) and the Domain Model (RD.065).

Current Development Process Model Example

An organization could use the following process to manage development of applications and their integrations. Note that this example uses roles as described in the task, Document Present and Future Organization Structures (RD.012).


Business Analyst
Solution Architect
Integration Architect
1. Demand Conceptual Specification
2. Demand technical specification
4. Check repository for reusability of existing services
3. SOA Integration?
yes
no
6. Approve?
no
7. Reject Demand
yes
8. Chose supplier
yes
5. Approve?
no
9. Develop
Testing and Deployment Engineer
10. Testing
no
12. Approve?
11. Tests Passed?
no
yes (integration project)
yes
yes (non-integration project)
13. Deployment
14. Monitor

Activities 1 through 14 are described below:

No.	Activity Name	Description
1.	Demand Conceptual Specification	Business analyst performs initial analysis of a demand and produces a conceptual design.
2.	Demand Technical Specification	Solution architect analyzes the initial documentation produced by the business analyst. The solution architect produces technical specification for the development that includes specification of interfaces/services and other assets in case of integration.
3.	SOA Integration?	If the development involves integration of two or more applications and the solution architect decides to follow the SOA integration style, he/she contacts the Integration Architecture team. The solution architect may also choose to follow the point-to-point integration and communicate with owners of involved applications directly.
4.	Check Repository for Reusability of Existing Services	Integration architect performs an analysis of a technical specification such as he/she checks existing assets in the service repository for their reusability.
5.	Approve?	Integration architect approves or rejects the integration.
6.	Approve?	The solution architect approves or rejects the demand.
7.	Reject Demand	
8.	Choose Supplier	The solution architect chooses a supplier for implementation (external company or in-house development).
9.	Develop	The solution architect monitors and checks the development.
10.	Testing	When implementation is ready, the tester performs tests.
11.	Tests Passed?	If tests fail, the tester requests reimplementation/fixing of any failed functionality (activity 9) otherwise it requests approval to deploy the integration.
12.	Approve?	The integration architect approves or rejects the integration. When rejected, the development must correct any issues.
13.	Deployment	The system administrator deploys the solution.
14.	Monitor	Integration architect monitors the performance of integration.

Current Governance Process Example

An organization could use following process to make sure that information in an offline service repository in an Excel spreadsheet is up-to-date.

- Repository owner extracts the list of services of an application from the latest version of the repository (the master repository) to the Excel spreadsheet and sends them by email to the owner of the application (the application owner).
- Application owner checks the information about services, validate the information if not valid or up-to-date and/or adds new services that are not in the list.
- Application owner contacts the repository owner if he/she wants to add a new protocol, transport or consuming application not in the list of protocols/applications.
- Application owner sends the updated list back to the repository owner.
- Repository owner checks for the overall consistency of the information provided by the application owner and clarify any issues with him/her.
- Repository owner updates the master repository with the updated information.

Back to Top

RD.140 Create Requirements Specification

In this task, you document requirements for SOA Governance. Take into account issues that you encounter when documenting the following:

- organizational structures in the the Present and Future Organization Structures (RD.012)

- overall high-level view on the organization's systems related to SOA Governance and an envisioned integration of SOA Governance tooling support in the Technical Architecture Requirements and Strategy (TA.020)
- Domain Model (RD.065)
- Current Process Models (RD.030)

By analyzing issues, you create a list of requirements.

For example, you may encounter following issues:

- There may be duplicate information in service metadata and WSDL description.
- There is no versioning of services.
- Solution architects may decide on an ad hoc basis about integration of applications as a response to a business requirement without involving the Integration Architecture team. As a result, they integrate applications in a point-to-point style and on an ad hoc basis.
- There is no versioning of services. An update of a service may cause non-functioning integration with service's consumers.
- The propagation of changes in services' interfaces in the offline service repository is slow thus service consumers cannot react quickly on changes in the environment.
- It is not clear under which conditions the integration is done in a point-to-point style and when the integration should follow the SOA approach.
- It is not clear how stakeholders use and should use the offline service repository.

Back to Top

RD.045 Prioritize Requirements (MoSCoW)

In this task, you prioritize the requirements identified from the Data Acquisition and Conversion Requirements (CV,010) and the Requirements Specification (RD.140), using the MoSCoW technique.

Back to Top

RD.042 Develop Glossary

The role of SOA Governance, among other things, is to establish an enterprise-wide understanding of concepts and terms that all SOA projects use. In this task, you develop this glossary. Start with the Glossary in the SOA Governance Implementation Overview.

Back to Top

TG.SGID SOA GOVERANCE IMPLEMENTATION DESIGN GUIDANCE

DS.090 Design Data

Typically, you implement SOA Governance support using an Enterprise Repository that already has pre-defined asset types that you can use out of the box. In addition, you may have the organization's legacy repository and its structures to describe services and applications.

In this task, you design data structures for asset types, their relationships, categorizations of assets, various value lists, and users and roles.

Asset Types and Relationships

Design data structures for asset types that will include the organization's legacy repository and Enterprise Repository's out-of-the-box structures. As an input, use the

Domain Model (RD.065) and the documentation for the Enterprise Repository describing the asset types it supports. Make sure to follow the Enterprise Repository's best practices for customizations.

Depending on your Enterprise Repository, you may adopt certain rules. For example, when you develop structures for asset types, you take respective Enterprise Repository structures as a starting point and make any customizations to it when necessary. Before you create any such customizations, try to map specific structures to structures provided by the Enterprise Repository. This allows you to limit the level of customizations, which in turn may help you to manage future upgrades of the Enterprise Repository software to newer versions.

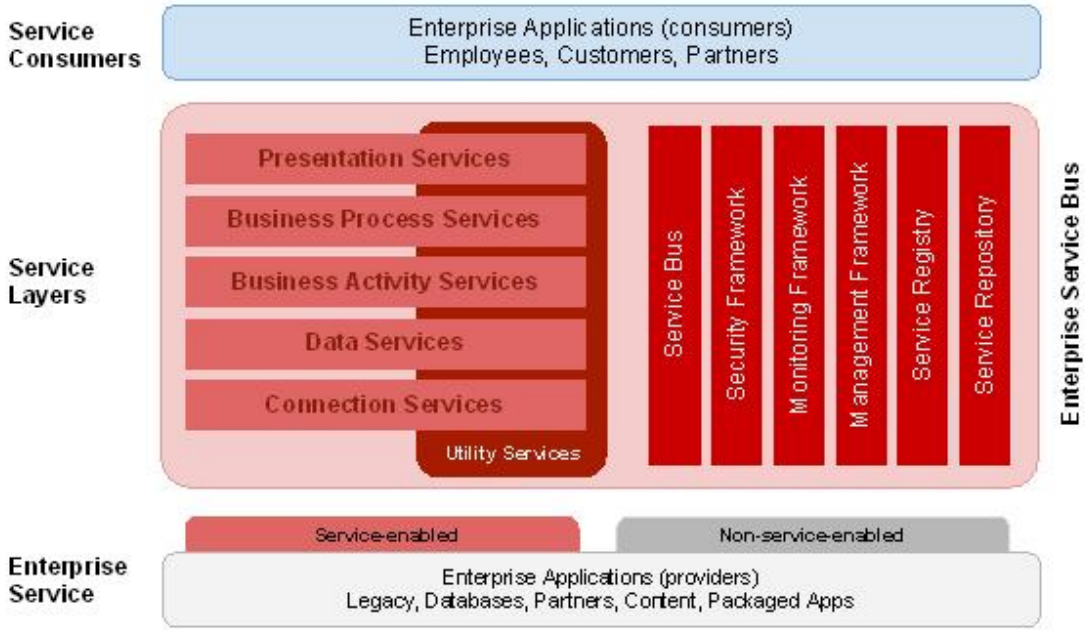
As an example, you could create two asset types, namely Service and Application and two relationships as shown in the following figure. PP denotes a prefix you assign to the asset type that uniquely identifies the organization. This allows you to distinguish asset types that you customize with pre-defined asset types that you use out-of-the-box.

For every asset type, you should further specify structure. As noted above, take the Enterprise Repository structure as a starting point and add or remove specific properties according to your needs. Further, you should specify mapping rules to map from legacy repository's structures to the new structures.

Categorizations of Assets

In order to help Enterprise Repository's users to effectively search for assets, an Enterprise Repository typically provides a means to classify assets according to various "views." For example, an Enterprise Repository may provide pre-defined views, such as, service layers that classify services according to client-server architectural view on the enterprise architecture, Line of Business (LOB) that classify assets based on the domain where assets operate (such as IT, financial, production, etc.). When designing categorization schemas for assets, you should start with pre-defined categorizations provided by the Enterprise Repository and add or modify any categorizations according to the organization's needs.

One of the most important categorization schemas used to classify services are service layers. The following figure shows Oracle's Enterprise Architecture Program (OEAP) that defines industry standards for service layers.



When you convert service descriptions from legacy service repository, you should also specify how you map information from this repository to new service layers.

When you migrate services from a legacy service repository, you may further specify rules that you apply when mapping information from this repository to new service layers. In other words, you should be able to identify available information in the legacy service repository to identify a service layer for services that you migrate. On the other hand, you may only be able to distinguish enterprise services and business activity services in your legacy service repository as there might not be any relevant information to distinguish between, for example, business activity services and presentation services. To illustrate, you could define following rules:

- Any service provided by an application that represents an Enterprise Service Bus in your legacy service repository will be classified as the Business Activity Service.
- When a service description contains information about the SOAP protocol and the providing application is not an Enterprise Service Bus, the service will be classified as service-enabled enterprise service. Otherwise, it will be classified as non-service enabled enterprise service.

Users and Roles

When you design asset types, you also specify users and their roles that define access rights for assets and responsibilities within the service lifecycle. Use the list of stakeholders from the Present and Future Organization Structures (RD.012) as a starting point for this task.

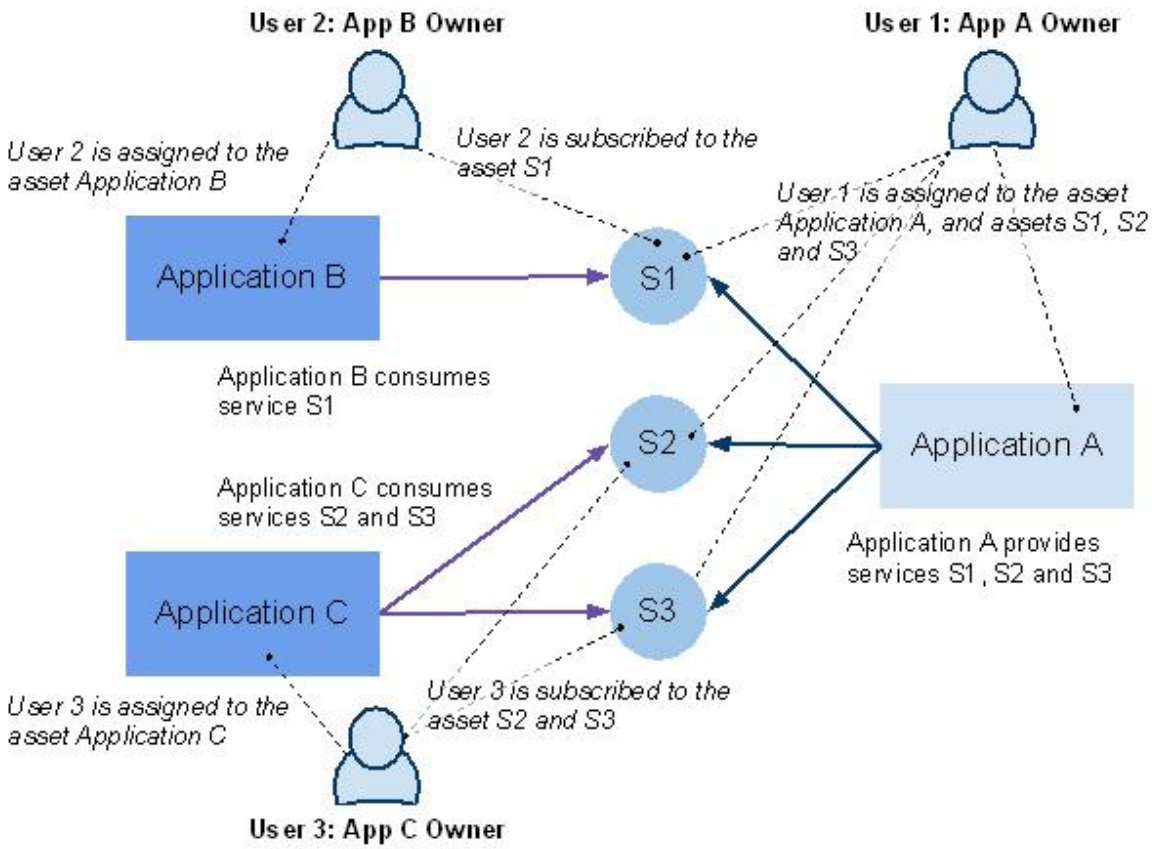
Consider the following example. Depending on your Enterprise Repository capabilities, you could define the following roles and their access rights:

- **Registrar** – owner of the Enterprise Repository. The registrar has all view and edit rights in the Enterprise Repository – that is, to edit assets, asset types, their categorizations, as well as perform approval actions within the service. Typically, a member of the Integration Architecture Department has this role.
- **Advanced Submitter** – owner of an application. The advanced submitter has rights to view and edit assets; however, he/she cannot define the asset type structures. Typically, a member of the Solution Architecture Department has this role.
- **Viewer** – read only role. Viewer can only view existing assets but cannot modify them. Typically, a member of the Business Analysis Department has this role.

Moreover, your Enterprise Repository may allow you to define at least two explicit relationships between a user and an asset namely assign and subscribe, and you should define the meaning of those relationships. For example, you can specify following rules:

- When a user is assigned to an asset, it means that he/she is responsible for the asset's content, its relationships to other assets, etc. By default, owners of assets (such as owners of applications) are assigned to these assets. Registrar can modify assignment to assets.
- When a user is subscribed to an asset it means that he/she is generally interested in any changes in the asset's definition, description and usage. By default, a user who owns an application that consumes services is subscribed to those services.

The following figure further illustrates the above rules:



Back to Top

DS.100 Design Behavior

In this task, you design the SOA Governance processes that may include the migration process and service lifecycle process. The goal of this task is to make sure that the organization's SOA Governance processes (i.e., processes you analyzed in task RD.030) are supported by the Enterprise Repository.

The processes described herein use the following color conventions.

- **Gray** – built-in states and service lifecycle states
- **Blue** – activities of the solution architect (Advanced Submitter role)
- **Red** – activities of the integration architect (Registrar role)
- **Cyan** – activities of the tester and the system administrator (Advanced Submitter role)

Service Lifecycle States and Enterprise Repository Support

When you design the service lifecycle process, use the development process from the Current Process Model (RD.030) as the input and define the states in which an asset can occur within this process. Using the development process example from RD.030 as the input, you could define the following service lifecycle states:

State	Description
A. Propose:	An asset has been submitted for review.
B. Plan:	The proposed asset has been approved, the development is planned.
C. Build:	The asset has been built.
D. Test:	The asset has been tested.
E. Release:	The development of the asset passed tests, the implementation was approved and the asset is released in the production environment.
F. Retire:	The asset is retired and is no longer used.

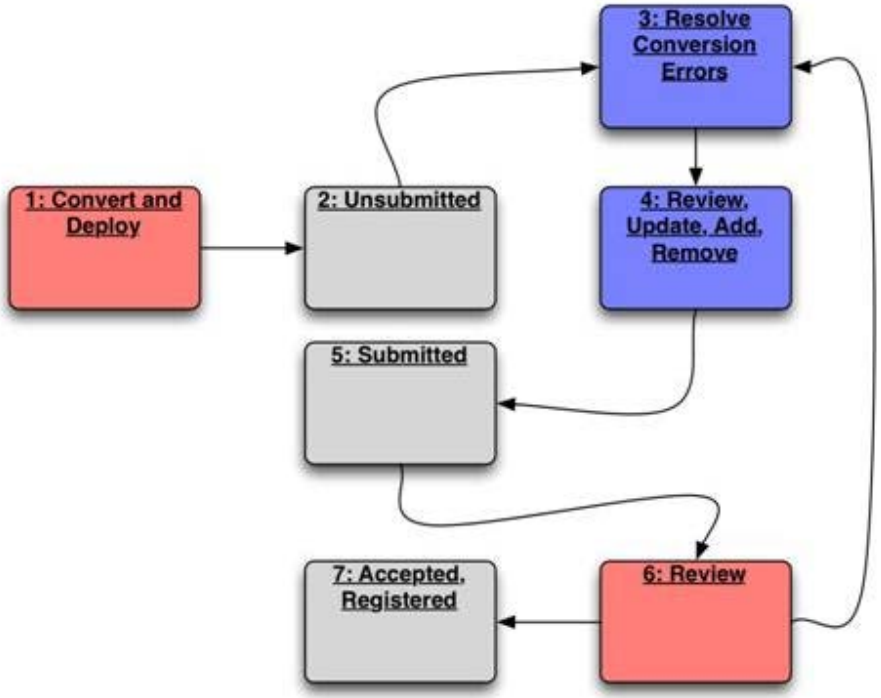
The Enterprise Repository should provide support for the service lifecycle process that uses the above states. For this purpose, the Enterprise Repository may use a built-in process for the asset management that may define built-in states as follows:

- **Unsubmitted** – The asset is under construction.
- **Submitted** – The asset was submitted and ready for review.
- **Accepted** – The asset was accepted and further work can be processed on the asset.
- **Registered** – The asset is valid and ready to be reused.

Note that the built-in process of the Enterprise Repository cannot usually be customized. The goal of the Behavior Design for SOA Governance is to implement development processes with defined service lifecycle states and to use the Enterprise Repository built-in process to support the service lifecycle.

Migration Process

The goal of the migration process is to move all assets from the legacy repository to a fully fledged Enterprise Repository and to make sure that all assets are valid and up-to-date. You can use the Enterprise Repository built-in process to manage the migration process. The following figure shows an example of this process depicted as a Control State Diagram that defines the activities of the migration process together with the Enterprise Repository built-in states.



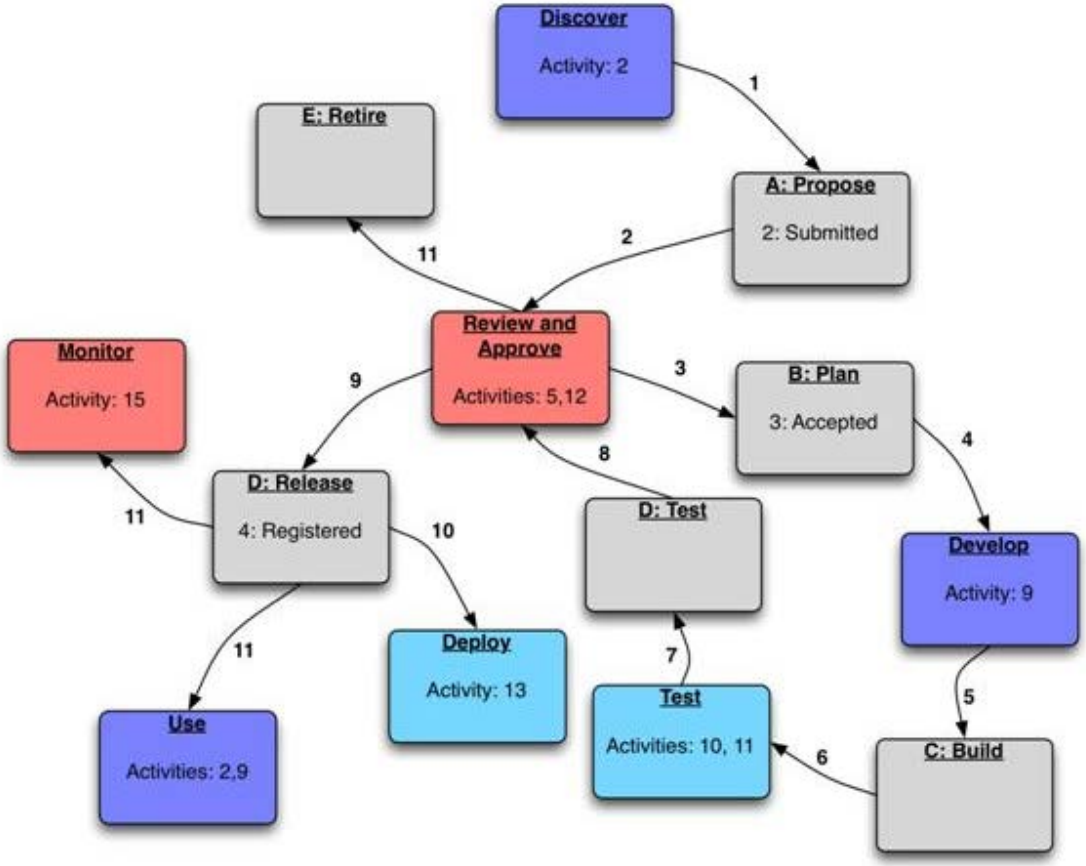
The following describes the activities in the above process:

- **Develop and Convert (administrator roles) (Activity 1):** The integration architect develops Enterprise Repository asset types and runs conversion utilities to convert assets from legacy repository to the new Enterprise Repository structures. All converted assets will get to the unsubmitted state in the Enterprise Repository. This means that all assets were converted and they are ready to be reviewed by asset owners. For details on the conversion, see the task, Design Conversion Components (CV.040).
- **Resolve Conversion Errors (Advanced Submitter) (Activity 3):** The owner of the asset resolves all conversion warnings that were generated from the conversion (Activity 1). The owner further reviews the asset data. He/she may edit it, that is, update the data, add new information, delete any information or delete the asset. After that, he/she changes the asset's state to submitted.
- **Review (Registrar) (Activity 6):** The registrar reviews each asset in the submitted state. If the registrar agrees with all the information of the asset, he/she will perform following actions. If the asset service lifecycle state (customizable states) is other than "Release," the registrar will set the asset's built-in state to "accepted." If the state is "Release," the registrar will set the asset's built-in state to "registered." The service lifecycle process (see the next section) may follow up on this with an activity that corresponds to the asset state.

Service Lifecycle Process

The goal of the service lifecycle process is to manage the lifecycle of assets with help of the Enterprise Repository. In particular, you take the development process the Current Process Model (RD.030) as the input and you use the Enterprise Repository's built-in states and workflows to support this process.

The following figure (and subsequent activity description) shows how the Enterprise Repository supports the development process example from RD.030:



- **Discover (Activity 2):** The solution architect tries to discover assets that relate to the planned development. If they cannot find an asset for reuse and a new asset should be developed, the solution architect submits a proposal for the new asset to the Enterprise Repository. The solution architect should set the service lifecycle state to “Planned” and built-in state to “submitted.”
- **Review and Approve (Activity 5):** The integration architect reviews any proposed assets submitted by the solution architect and either rejects them (not shown in Figure) or approves them. When the integration architect approves the asset, they set the service lifecycle state to “Plan” and the built-in state to “accepted.” The asset can now be developed.
- **Develop (Activity 9):** The solution architect plans the development (such as, chooses the supplier, develops detail specification for implementation, etc.). When the Development team finishes building a solution, the solution architect should change the service lifecycle state to “Build” (transition 5) and pass the solution to testing (transition 6). The built-in state remains the same.
- **Test (Activities 10, 11):** The tester performs tests. If the tests pass, they change the service lifecycle state to “Test” (transitions 7, 8). The built-in state remains the same.
- **Review and Approve (Activity 12):** The integration architect approves the solution and related assets for deployment and sets the service lifecycle state to “Release” (transition 9). The integration architect also sets the built-in state to “Registered.”
- **Deploy (Activity 13):** The system administrator deploys the solution to the production environment.
- **Monitor (Activity 14):** The integration architect monitors the solution in the running environment.
- **Use (Activities 2, 9):** The solution architect and developers use assets when developing a new solution.

Valid Combinations of Service Lifecycle and Built-In States

When you design the service lifecycle process and the applicable Enterprise Repository support, you should clarify the valid combination of service lifecycle states and

built-in states. The following table shows example combinations of states that may occur during the migration and service lifecycle processes.

Lifecycle State(s)	Built-In State	Description
Propose	Submitted	Solution architect proposed as asset.
Plan	Accepted	The accepted asset by the registrar is being planned to be developed.
Build	Accepted	The accepted asset by the registrar was built.
Test	Accepted	The accepted asset by the registrar was tested.
Release	Registered	The asset was released and registered by the registrar.
Retire	Registered	The asset was retired; however, it can be reused.
Propose, Release	Accepted	This is an invalid combination of states.
Propose, Plan, Build, Test	Registered	This is an invalid combination of states.
Propose, Plan, Build, Test, Release, Retire	Unsubmitted	Asset created as a result of conversion. The asset awaits review by the asset owner.
Propose, Plan, Build, Test, Release, Retire	Submitted	Asset owner reviewed the asset after conversion.

Back to Top

DS.110 Design Business Rules

When you design the Domain Model (RD.065) and processes around that model (RD.030), you should clarify the rules that determine your Enterprise Repository's consistent state.

Example Business Rules

Once again using the development process example from RD.030, you could create the following business rules:

> Every application must provide at least one service and vice-versa and all provided and consumed asset services must be of the type PP Service.
>
> Every application must provide at least one service and vice-versa.
>
> Every SOAP service must have a valid WSDL defined. Both the WSDL must be a valid URL and HTTP GET must obtain the WSDL document.
>
> Every application must have a user responsible for it. The user comes from a department that owns the application.

Ensuring the Enterprise Repository's Consistent State

The way you implement the business rules depends on your Enterprise Repository technology. For example, an Enterprise Repository that allows you to develop a custom meta model for asset types may not directly enforce users to provide valid data. In particular, if a user does not know the data to provide for an asset description, the user is allowed to leave the data blank. Moreover, meta data structures in the Enterprise Repository may not allow you to define any hard cross-asset rules. For example, it might not be possible to define cardinality constraint rules between an application asset and a service asset such that the application must always provide at least one service (in this case, the structures may exist in the Enterprise Repository meta model thus such constraints cannot be enforced by means of the database cardinality constraints).

Depending on your Enterprise Repository technology, you should determine how the business rules will be ensured. For example, you may create automated reports that check for the Enterprise Repository's consistency and generate warnings, or you use Enterprise Repository's extensions that allow you to automate and customize workflows.

Back to Top

CV.040 Design Conversion Components

In this task, you design conversion components in order to populate the Enterprise Repository with existing assets available in the organization's environment. The conversion components should cover:

- harvesting from existing middleware platforms, and
- conversion of data in the legacy repository to the new Enterprise Repository.

Conversion

Define components and a process to perform conversion. This is where you describe the activity of the migration process (see task, DS.100 Design Behavior – Migration Process). When you design the components and the process, you should:

- Break down the conversion to a number of components that you can run one by one. In this way, you can reduce the complexity of the whole conversion process as well as aid in resolving issues.
- Design the components in a way that you meet a requirement of a "continuous conversion," that is, you can always start the conversion process from the initial state or in any newly installed environment. This will help you to perform the whole conversion iteratively. Also, if you encounter any errors in mappings, conversion logic, etc., that may corrupt the target database; you will always be able to start the processes all over again from the initial state.

Note that the conversion components are used to run a conversion process. Typically, you perform this process in conjunction with a development process of the Enterprise Repository database in which you design asset structures. Use Enterprise Repository tools to create such structures. You should then export the structures from the repository so that you can later use them in the conversion process as the structure for the target database.

Consider designing the following conversion components:

- **Create Initial Database** – This component should delete all content of the Enterprise Repository database and create the initial one.
- **Setup the Database** – This component will setup the database such as with user roles, default access rights, etc.
- **Import Structures** – This component should import the target database structures (exported from the Enterprise Repository development process).
- **Check the Database** – This component will check the target database for any rules that may be violated and generates warnings or errors (see task, DS.110).
- **Clean the Environment** – This component will clean the database for any unnecessary information, such as, events generated from the conversion, etc.

As noted earlier, you may iteratively build the target database by performing development and conversion until you reach the state when all data from the legacy repository is converted to the Enterprise Repository. After that, perform the additional activities of the migration process as described in the Migration Process section of task, Design Behavior (DS.100).

Harvesting

In this task you design components for harvesting of assets from running middleware in your environment. When you design components for harvesting, perform the following steps:

1. Reuse any components that your Enterprise Repository may provide as solution packs for harvesting of assets from supported middleware platforms.
2. Develop new components, if necessary, or extend existing ones.
3. Determine sources of information from which to harvest the assets, such as, version control repositories (e.g., SVN, CVS), middleware project files, running middleware's meta data databases, etc.
4. Define configurations, including locations of sources, credentials that the harvester will use to connect to the source, a frequency of running the harvester, subsets of assets to be harvested, etc.

Back to Top

TR.070 Develop User Learning Plan

SOA Governance should define all important structures and processes for effective governance of the SOA Implementation projects in the organization. However, it is important to note that in order to be successful and widely adopted in the organization; the processes must be implemented and followed by the appropriate departments and their staff members. The departments' staff members should be trained on how to use the Enterprise Repository as well as in the Governance processes.

When your SOA Governance Implementation project also uses Enterprise Repository to perform migration from the legacy repository to the new Enterprise Repository see (the Migration Process section of task, Design Behavior (DS.100), it may be useful to plan at lest two training workshops as follows:

Workshop 1

In the first workshop, users should become familiar with the Enterprise Repository concepts, its specific structures and the processes developed for SOA Governance in your organization. The agenda for this workshop should cover following topics:

- Welcome and Workshop Overview (15 minutes)
- General Introduction to SOA Governance (60 minutes)
- Lab 1: Basic Training with Enterprise Repository Tool (60 minutes)
- Overview of the SOA Governance Project in the Organization (30 minutes)
- Lab 2: Migration Process Training (60 minutes)
- Conclusion, Next Steps

The workshop should first give an introduction to SOA Governance in general, train staff members on the basics of the Enterprise Repository functionality. Further, the workshop should give details on the SOA Governance project in the organization, explain the major processes and then in detail show the migration process. Users should get hands-on training on how to work within the migration process to resolve the issues that occur during the migration. They should receive enough knowledge to be able to finish the process after the workshop.

Workshop 2

In the second workshop, users should become familiar with service lifecycle process and train with a scenario in which they develop services, register them as assets and reuse them in development processes of applications. In addition, the trainer should verify whether users understood the migration process from Workshop 1 and discuss any issues they might have with the Enterprise Repository or any processes.

Back to Top

TR.080 Develop User Learningware

In this task, you develop any learning materials for the workshops defined in task, Develop User Training Plan (TR.070). The materials may include presentations and materials for labs as follows:

- Presentation: Include a general introduction to SOA, SOA Governance, its position towards SOA, introduction to your Enterprise Repository technology and its main functionalities.
- Presentation: Provide a specific implementation of the Enterprise Repository in the organization.
- Lab: Provide a step-by-step description of activities for users to train the basic functionality of the Enterprise Repository technology.
- Lab: Provide a step-by-step description of activities for users to train resolving migration issues.

Back to Top

TA.020.2 Define Technical Architecture Requirements and Strategy

In this task, you should design the technical requirements, processes and technologies for the testing and production environments to run SOA Governance processes in the organization.

When designing the Technical Architecture, consider the following points:

- all components that are required to build the whole environment such as Application servers, Web server, Enterprise Repository application, database, reporting server, etc.
- configuration of products, versions, installation paths, configurations of application server domains, configuration of Web servers
- security configurations, such as, single sign-on, LDAP servers, etc.
- application server endpoints where users can access applications, such as, administration console, main Enterprise Repository application console, Reporting server, etc. and credentials to access the system
- backup and recovery strategy for Enterprise Repository data

Back to Top

TA.040 Define Reporting and Information Access Strategy

In this task, you define any reports required for SOA Governance. In particular, you may define two kinds of reports: operation reports and decisions support reports.

Operation reports are required for daily operation of the business. They may also facilitate your migration processes and may include information on the number of warnings or errors generated from the migration process that need to be resolved by owners of assets. **Decision support reports**, on the other hand, facilitate the business through measurements and prediction of business performance. The Enterprise Repository may already include a set of pre-defined decision support and operational reports that you should further adjust to the organization`s needs.

Back to Top

ORACLE

Copyright © 2006, 2016, Oracle and/or its affiliates. All rights reserved.